Übungsblatt 2: Speicherhierarchie und Caches

Mikrocomputertechnik 3 (MCT3)

Musterlösung

Zielsetzung

Sie vertiefen Vorlesung 2: Lokalitätsprinzip, AMAT, Cache-Mapping (Direct Mapped / Set-Associative), Schreibstrategien und cache-freundlichen C-Code (Row-Major vs. Column-Major).

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 14 P, Aufgabe 2: 8 P (Gesamt: 22 P)

Teil 1: Theorie — Speicherhierarchie und Caches

Aufgabe 1.1: Das Lokalitätsprinzip

Caches funktionieren nur, weil Software kein völlig zufälliges Zugriffsverhalten zeigt.

1. Erklären Sie den Unterschied zwischen zeitlicher Lokalität (Temporal Locality) und räumlicher Lokalität (Spatial Locality).
2. Warum sind Arrays aus Sicht des Caches typischerweise effizienter zu verarbeiten als verkettete Listen (Linked Lists)?

Musterlösung Aufgabe 1.1

1. **Zeitliche Lokalität:** Eine kürzlich genutzte Adresse wird mit hoher Wahrscheinlichkeit bald erneut benötigt (z. B. Schleifenzähler, Instruktionen in einer Schleife) — der Cache behält solche Daten.
   Räumliche Lokalität: Nach Zugriff auf eine Adresse folgen oft Zugriffe auf benachbarte Adressen — deshalb lädt die Hardware bei einem Miss eine ganze Cache-Line (z. B. 64 Byte).

2. Arrays liegen als zusammenhängender Block im Speicher. Ein Miss beim ersten Element zieht Nachbarn mit in den schnellen SRAM. Bei verketteten Listen liegen Knoten oft weit verstreut (`malloc`); mitgeladene Nachbardaten sind dann meist nutzlos → viele Misses.

Aufgabe 1.2: AMAT-Berechnung

Die Average Memory Access Time (AMAT) ist eine zentrale Kenngröße der Speicherarchitektur.

Ein Prozessor habe:

- Hit Time des L1-Caches: 1 Takt
- Miss Penalty (Zugriff Hauptspeicher/DRAM): 100 Takte
- Hit Rate des L1 bei einem Programm: 95 %

1. Berechnen Sie die AMAT. Geben Sie Formel und Rechenweg an.
2. Welche Auswirkungen hat es auf die AMAT, wenn Pointer-Ketten die Hit Rate auf 50 % senken?

Musterlösung Aufgabe 1.2

Formel (ein Cache-Level, Miss geht zum Hauptspeicher):

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

1. Miss Rate $= 1 - 0{,}95 = 0{,}05$

$$\text{AMAT} = 1 + (0{,}05 \times 100) = 1 + 5 = 6 \text{ Takte}$$

2. Bei Hit Rate 50 %: Miss Rate $= 0{,}5$

$$\text{AMAT} = 1 + (0{,}5 \times 100) = 1 + 50 = 51 \text{ Takte}$$

Die mittlere Zugriffszeit steigt stark — schlechte Lokalität verlangsamt das Programm deutlich.

Aufgabe 1.3: Cache-Mapping und Adresszerlegung

Eine 32-Bit-Adresse wird in Tag, Index und Offset zerlegt.

1. Welche Aufgabe haben Tag, Index und Offset beim Lookup im Cache?
2. Erklären Sie Thrashing bei einem Direct-Mapped-Cache anhand zweier Arrays A und B.
3. Wie mildert ein N-Way Set-Associative Cache Thrashing? Warum sinkt dabei typischerweise die Zahl der Index-Bits?

Musterlösung Aufgabe 1.3

1. **Offset:** unterste Bits — Byte-Position innerhalb der Cache-Line.
   Index: mittlere Bits — wählt die Zeile bzw. das Set.
   Tag: oberste Bits — Vergleich mit dem gespeicherten Tag, ob die Line die gesuchte Adresse trägt.

2. **Thrashing:** A und B kollidieren auf demselben Index. Im Direct-Mapped-Cache gibt es nur einen Platz pro Index — abwechselnde Zugriffe werfen sich gegenseitig heraus. Die Miss Rate kann nahe 100 % liegen, obwohl andere Lines frei sind.

3. **Set-Associative:** Mehrere Ways pro Set. A und B können denselben Set-Index teilen, aber in unterschiedlichen Ways liegen. Bei fester Cache-Größe und fester Line-Größe bedeutet mehr Associativity weniger Sets → weniger Index-Bits; die Bits wandern typisch ins Tag.

Aufgabe 1.4: Schreibstrategien

Beim Schreiben droht Inkonsistenz zwischen L1-Cache und RAM.

1. Vergleichen Sie Write-Through und Write-Back hinsichtlich Bus-Belastung und Performance.
2. Welche Rolle hat das Dirty-Bit bei Write-Back, und wann werden Daten in den RAM geschrieben (Eviction)?

Musterlösung Aufgabe 1.4

1. **Write-Through:** Jeder Write geht (nahezu) synchron in Cache und Hauptspeicher → hohe Bus-Last, oft langsamere Stores.
   Write-Back: Write zunächst nur im Cache → hohe Performance, geringe Bus-Last; RAM kann vorübergehend veraltet sein.
2. Das **Dirty-Bit** markiert eine veränderte Line. Zurückschreiben in den DRAM erfolgt typisch bei **Eviction** (Verdrängung), wenn der Platz für eine neue Line benötigt wird — nicht bei jedem Write.

Teil 2: Praxis — Cache-freundlicher C-Code

Aufgabe 2.1: Row-Major vs. Column-Major

Gegeben sind zwei Snippets, die ein 1000×1000-Integer-Array auf 0 setzen. C speichert 2D-Arrays **zeilenweise** (Row-Major).

Variante A:

```
for (int row = 0; row < 1000; row++) {
    for (int col = 0; col < 1000; col++) {
        bild[row][col] = 0;
    }
}
```

Variante B:

```
for (int col = 0; col < 1000; col++) {
    for (int row = 0; row < 1000; row++) {
        bild[row][col] = 0;
    }
}
```

1. Welche Variante ist cache-freundlich, welche problematisch? Begründen Sie physikalisch (Lokalität, Cache-Lines).
2. Was passiert in Variante B bei jedem Schritt der inneren Schleife bezüglich der geladenen Cache-Line?

Musterlösung Aufgabe 2.1

1. **Variante A** (innere Schleife über Spalten einer Zeile) ist cache-freundlich: aufeinanderfolgende Elemente liegen im RAM nebeneinander → räumliche Lokalität.
   Variante B (innere Schleife über Zeilen einer Spalte) springt um jeweils eine Zeilenbreite (hier $1000 \times$ sizeof(int) Byte) → schlechte räumliche Lokalität, viele Misses.

2. Bei B lädt ein Miss zu `bild[r][c]` typisch eine Line mit horizontalen Nachbarn in derselben Zeile. Der nächste Zugriff `bild[r+1][c]` liegt weit entfernt — die mitgeladenen Nachbarn werden oft nicht genutzt, bevor die Line verdrängt wird. Folge: nahezu Miss pro Innenschritt (je nach Line-Größe und Array-Layout).

Aufgabe 2.2: Messung (optional / Labor)

Übertragen Sie Variante A (und zum Vergleich Variante B) in einen Simulator mit Daten-Cache (z. B. Ripes). Aktivieren Sie den D-Cache und vergleichen Sie die Hit-Rates.

Musterlösung Aufgabe 2.2

Erwartung: Variante A ist deutlich schneller und zeigt eine höhere Hit-Rate als Variante B — wegen der räumlichen Lokalität im Row-Major-Layout. Die absolute Rate hängt von Line-Größe, Cache-Kapazität und Associativity ab; entscheidend ist der relative Unterschied.